

תיעוד נלווה לפרויקט בקומפילציה (ממ"ן 16)

מגיש: אורן פאליי | ת"ז: 215579913

שיקולי מימוש

לפרויקט זה השתמשתי בכלי Flex ו-Bison כדי ליצור מנתח לקסיקלי ותחבירי בהתאמה. תהליך הקומפילציה כולל המרה של הקוד לעץ תחביר מופשט (AST), עליו מתבצע ניתוח סמנטי ולאחר מכן הפקת קוד ביניים בשפת Quad.

המנתח הלקסיקלי (Lexer)

-קובץ lexer.l משמש להגדרת האסימונים של שפת CPL, בהתאם לחוקי הדקדוק שלה. אחד המרכיבים המרכזיים במימוש זה הוא התמיכה בליטרלים מסוגים שונים — ליתר דיוק, ליטרלים מסוג int ו-float. שני הסוגים הללו מזוהים תחת אותו טוקן, NUM, אך נבדלים זה מזה בערך שדה type, המתווסף לכל טוקן ומאפשר להבחין ביניהם בשלב התחבירי.

- המנתח הלקסיקלי מייחס ערך סמנטי (yylval) כמעט לכל אסימון. הדבר נעשה במיוחד כאשר לאסימון יש מספר תתי-סוגים — למשל, אופרטורים, תנאים או מילות מפתח שונות. הערך הסמנטי כולל מידע חשוב כמו הסוג של הטוקן ולעיתים גם ערך מספרי ממשי במקרה של ליטרלים.

- לצורך מימוש התכונה הזו, נעשה שימוש במבני enum נפרדים עבור כל סוג טוקן שיכול לקבל מספר אפשרויות. כך ניתן להבטיח מיפוי חד-חד ערכי בין טוקנים לבין המשמעות שלהם במנתח התחבירי.

- כאשר המנתח הלקסיקלי נתקל בתו שאינו שייך לשום אסימון מוכר, תודפס הודעת שגיאה ברורה הכוללת את מספר השורה בה אירעה השגיאה ואת התו שגרם לה. התראה זו מקלה על איתור תקלות בקוד המקור בשלב מוקדם של הקומפילציה.

המנתח התחבירי (Parser)

לבניית המנתח התחבירי בפרויקט בחרתי להשתמש ב-Bison, תפקיד המנתח התחבירי יהיה להמיר את הקלט לעץ תחבירי אבסטרקטי (AST). הקובץ parser.y מכיל את כללי הדקדוק של שפת CPL, ולפיהם Bison יוצר את המנתח התחבירי. תהליך הפירסונג מתבצע באופן הדרגתי: בכל שלב נשלף האסימון הבא באמצעות הקריאה לפונקציה (yylex).

- לכל כלל תחבירי מוגדרת פונקציית יצירה ייעודית בשם *_create, אשר אחראית על בניית המבנה המתאים בעץ התחבירי. כאשר יש כללים תחביריים בעלי מספר סוגים — לדוגמה Factor או BoolTerm — נשמר גם ערך המייצג את הסוג הספציפי, כדי לאפשר טיפול נכון והבחנה בהמשך התהליך.

- בנוסף, על מנת לאתר שגיאות תחביר בצורה יעילה, נעשה שימוש בתכונה פנימית של Bison אשר יוצרת באופן אוטומטי טוקן שגיאה (error) כאשר רצף האסימונים אינו תואם לכללים שהוגדרו. בעזרת תכונה זו ניתן להגדיר כללי דקדוק מיוחדים לטיפול בשגיאות תחביר, כך שהתהליך לא יקרוס אלא יתאושש מהשגיאה וימשיך בניתוח.

- בעת גילוי שגיאה, מופעלת הפונקציה yyerror, המדפיסה את מספר השורה. כדי למנוע עצירה מוחלטת של תהליך הקומפילציה, אני עושה שימוש ב-yyerrok, המאפשר המשך עיבוד גם לאחר טיפול בשגיאה אחת — כך ניתן לזהות ולדווח על שגיאות נוספות באותו קלט.

עץ תחביר מופשט (AST)

- בפרויקט זה מימשתי את עץ התחביר האבסטרקטי (AST) באמצעות מבני נתונים שונים, כאשר כל מבנה מייצג רכיב תחבירי בשפה — לדוגמה: Expression, Statement, Factor, Term ועוד מה שמאפשר לטפל בכל דבר באופן

נקודתי ומחלק את המטלה הגדולה להרבה קטנות. כל אחד מהרכיבים הללו נבנה באמצעות פונקציית יצירה ייעודית בפורמט *_create, כגון:

- create_factor_num
- create_switch_stmt
- create_boolfactor_rel

- די לשמור על עקביות טיפוסית בין שלבי הקומפילציה, נעשה שימוש בפונקציות עיטוף כמו wrap_factor_as_expression ו-wrap_if_needed. פונקציות אלו משמשות למקרים בהם יש צורך לעטוף טיפוס אחד בטיפוס אחר באופן עקבי, בהתאם להקשר התחבירי שבו הוא מופיע.

עם סיום שלב הניתוח התחבירי, מתבצע מעבר נוסף על עץ ה-AST. בשלב זה, עבור כל סוג מבנה, קיימת פונקציית *_handle המתאימה לסוג המבנה ומבצעת עליו עיבוד. הפונקציה האחראית מבצעת את הבדיקות הסמנטיות הנדרשות בהתאם לרכיב שבו מדובר, ולבסוף יוצרת את הקוד הביניים הרלוונטי בפורמט Quad.

הסיבה העיקרית ליצירת עבור כל מבנה בגזירה נNODE ב AST היא שכאמור זה מקל על המימוש כי אנחנו מחלקים את העבודה לחלקים בעבור כל מסנה יהיה טיפול יחודי לו ושדות שמתאימים לו ספציפית ובהתאם לNODE אנחנו גם נדע איזה בדיקות סמנטיות יש לעשות

כמו כן היתרון נוסף אם היינו רוצים להרחיב את הספה זה יהיה קל מאוד כי עלינו יהיה להוסיף NODE לכל מבנה נוסף ופשוט להתאים את הגזירות בהתאם לפי התנאים שנקבל

ניהול זיכרון

-במהלך מימוש הקומפיילר, נדרשתי להקצות ולנהל מבני זיכרון שונים אשר מלווים את תהליך הקומפילציה מתחילתו ועד סופו. להלן המרכיבים המרכזיים:

1. טבלת סמלים (symbol_table)

זוהי טבלת גיבוב (Hash Table) אשר שומרת את שמות המשתנים שהוגדרו בתוכנית יחד עם הטיפוסים שלהם (int, float וכו').

- מוקצית באמצעות הפונקציה create_table().
- משוחררת עם free_table() בתום השימוש.

2. טבלת תוויות (label_map)

טבלה זו משמשת למיפוי תוויות כמו תווית של לולאות ופקודות if switch-case למיקומים בקוד הביניים (Quad instructions).

- מבוססת גם היא על מבנה Hash Table.
 - מוקצית עם calloc, תומכת בהתרחבות דינמית.
 - משוחררת בעזרת free_label_map().
- מימשי את שניהם באמצעות טבלת גיבוב

3. מחסנית תוויות לולאה (loop_stack)

מבנה מחסנית (Stack) דינמי שנועד לשמור את תוויות הלולאות הפעילות, כולל לולאות while ובלוקים של switch.

- מוקצה עם init_loop_stack().
- משוחררת עם free_loop_stack() בסיום התהליך.

שחרור זיכרון כולל

עם סיום מהלך הקומפילציה, פונקציית handle_program() אחראית לניקוי כלל משאבי הזיכרון שהוקצו. כך מובטח שכל המבנים ששימשו במהלך האנליזה התחבירית, הסמנטית והפקת הקוד ינוקו בצורה מסודרת ולא ייוותר זיכרון דלוף (memory leak).

מבני נתונים

- הוראות Quad

בשלב יצירת הקוד הביניים, כל הוראה מיוצגת באמצעות מבנה בשם Instruction, הכולל את השדות הבאים:

- op – סוג הפעולה (אופקוד)
- arg1, arg2 – שני ארגומנטים אופציונליים
- result – יעד הפעולה (תוצאה)

כל ההוראות נשמרות ברשימה מקושרת הנקראת InstructionList, אשר משמרת את סדר ההוראות. מבנה זה שימושי במיוחד כאשר יש צורך לעקוב אחרי יעדי קפיצה, כמו בפקודות תנאים ולולאות – כיוון שהוא מאפשר מעבר על כל ההוראות לפי הסדר שבו נוצרו כמו כן נשמור את המספר פקודות העדכני ברשימה הדבר ישמש אותנו לחישוב יעדי הקפיצה.

- טבלת סמלים (Symbol Table)

בפרויקט זה, טבלת הסמלים ממומשת באמצעות טבלת גיבוב (Hash Table) המשתמשת ב־פרובינג לינארי (linear probing). הטבלה משמשת לשמירה על מזהים שהוגדרו בתוכנית (כמו שמות משתנים) יחד עם הטיפוס שלהם (int, float וכו'), תוך שמירה על יעילות בגישה והימנעות מהגדרות כפולות.

עקרונות המימוש:

- תהליך הגיבוב מתבצע על שם המזהה (*char) באמצעות פונקציית hash פשוטה.
- כאשר מתרחשת התנגשות (כלומר, משבצת בטבלה כבר תפוסה), נעשה שימוש בפרובינג לינארי – מחפשים את התא הפנוי הבא ברצף.
- שדה עזר [used] מציין אילו תאים תפוסים ואילו פנויים, דבר שמאפשר לולאות חיפוש סגורות ויעילות.
- הטבלה תומכת בהרחבה דינאמית (rehashing): כאשר שיעור התפוסה עובר את 75%, הכיבולת מוכפלת, וכל המידע ממוקם מחדש בטבלה גדולה יותר.
- ניסיון להכניס מזהה שכבר הוגדר יגרום לשגיאה – מנגנון זה מונע redeclaration.

יעילות החיפוש:

- החיפוש מתבצע בזמן ממוצע של $O(1)$ – ברוב המקרים, המזהה יאוחסן או יימצא מיידית לפי ערך הגיבוב.
- גם כאשר מתרחשות התנגשויות, השיטה של linear probing שומרת על ביצועים טובים, כל עוד יש מספיק מרווח בטבלה.
- ההרחבה הדינאמית מונעת מצב שבו תאים סמוכים "נדבקים", וכך נשמרת יעילות גבוהה גם בתוכניות גדולות.

- טבלת תוויות (Label Map)

באופן דומה לטבלת הסמלים, גם טבלת התוויות ממומשת כ־Hash Table עם probing לינארי.

- תפקידה למפות שמות של תוויות (למשל בלולאות וב־switch-case) לאינדקסים של שורות קוד (Quad instructions).
- הגיבוב מתבצע על המחרוזת של שם התווית, כאשר התנגשויות נפתרות באותו אופן – חיפוש לינארי עד למציאת תא פנוי.
- כמו בטבלת הסמלים, זמן החיפוש נשמר קצר בממוצע $O(1)$ תודות להרחבה הדינאמית.

הסיבה למימוש באמצעות טבלאות גיבוב: זה טיפול קל בהן קל להגדיל אותם דינמית ולטפל בהתנגשויות כלומר זה מבנה ספציפי יחודי של טבלת גיבוב שיצרתי לעצמי שנוח ספציפית לצרכים שלי ופשוט לישום באמצעות פונקציה בודדת לכל דרישה וכן נשמר החיפוש הקצר

תמיכה בלולאות ו-switch

מחסנית ללולאות ותמיכה ב-switch

לצורך ניהול נכון של לולאות (while) ובלוקים של switch-case, נעשה שימוש במחסנית (loop_stack).

- המחסנית שומרת את התוויות הרלוונטיות בכל רגע בזמן בניית הקוד הביניים.
- כאשר נכנסים ללולאה חדשה או בלוק switch, התווית המתאימה נדחפת למחסנית.
- בסיום הלולאה/הבלוק, התווית נשלפת מהמחסנית.
- כך נשמרת היררכיה תקינה של תוויות, והקפיצות מתבצעות בדיוק למקום המתאים.
- הסיבה שבחרתי לממש כך היא ש:
- מבני switch ו-while יכולים להיות מקוננים זה בתוך זה.
- המחסנית מאפשרת לדחוף label חדש בתחילת המבנה, ולשלוף אותו כשמסיימים (LIFO).
כך תמיד break יתייחס למבנה הפעיל האחרון – בדיוק מה שאנחנו צריכים.

פונקציות חשובות בפרויקט שלי והסבר עליהן:

המערכת כוללת מספר פונקציות ליבה האחראיות על יצירת הקוד הביניים (Quad instructions), ניהול מזהים ותוויות, וכתיבת הפלט הסופי:

create_quad ♦

יוצרת הוראת Quad חדשה. הפונקציה מקבלת את סוג הפעולה (אופקוד) ושלושה פרמטרים: arg1, arg2 ו-result, ובונה מבנה Instruction מתאים.

append_quad ♦

מוסיפה את הוראת ה-Quad שנוצרה לרשימת הפקודות (InstructionList) באופן סדרתי, בהתאם לסדר הפקת הקוד.

new_label ו-new_temp ♦

- `new_temp`: מייצרת משתנה זמני חדש. השם שלו נקבע על ידי צירוף האות `t` עם מספר הריץ (למשל `t1`, `t2` וכו').

- `new_label`: מייצרת תווית חדשה, עם שם המבוסס על האות `l` ומספר סידורי (כגון `l1`, `l2` וכו').

המספור נקבע לפי מספר המשתנים/לייבלים שנוצרו עד כה, כך שאין חפיפות.

◆ `map_label`

מוסיפה את הלייבל לטבלת הלייבלים (`label_map`), שמממשת טבלת גיבוב. פעולה זו מאפשרת לשמור לכל תווית את המידע הנדרש – למשל, באיזו שורת `Quad` היא מופיעה.

◆ `set_label_line`

לאחר שממוקם לייבל בקוד, פונקציה זו מאתחלת את שדה מספר השורה המתאים בתוך מבנה הלייבל. היא משויכת לשלב שבו כבר ידוע היכן התווית אמורה להופיע ברשימת ההוראות.

◆ `patch_jumps`

פונקציה חיונית אשר נקראת לאחר סיום מעבר על עץ ה-`AST`. היא סורקת את רשימת הפקודות, ומאתרת את כל ההוראות מסוג `JMP` ו-`JMPZ`. עבור כל אחת מהן, היא מעדכנת את יעד הקפיצה בהתאם למספר השורה של התווית המתאימה כפי שנשמרה בטבלת הלייבלים.

◆ `write_quads_to_file`

פונקציה הסורקת את כל הוראות הקוד הביניים שברשימה, ומדפיסה אותן לקובץ פלט באופן סדרתי. במהלך ההדפסה, הפונקציה מתרגמת את קוד האופרטור (הנשמר כ-`enum` מספרי) לשם טקסטואלי של הפעולה (כגון `ADD`, `JMP`, `MUL` וכו').

הוראות הפעלה

הוספתי לתקיית הרצה קובץ `Makefile` בשביל להריץ יש לרשום את הפקודה `make` ולאחר מכן שורת ההפעלה תהיה `./cpq.exe<file_name>.ou`.